Inspire Hands Scripts Explanation

Python

python/ä, ä»£æ æ§å ¶ç¤ å″ /demo_485.py
Controls a 3rd-generation hand over RS-485 serial. It opens /dev/ttyUSB0, writes speed, force, and angle registers, reads back temperature and actual angle values, and triggers an action sequence. This is the simplest serial demo.

python/ä, ä»£æ æ§å ¶ç¤ å″ /demo_can.py
Controls a 3rd-generation hand using the CAN-style framed protocol. It builds custom frames with 0xAA ... 0x55, splits 6 values across two frames when needed, and reads values back in two parts.

python/å ä»£æ æ§å ¶ç¤ å″ /demo_485.py
Same structure as the 3rd-generation demo_485.py, but intended for the 4th-generation hand. Good starting point if the device is connected by USB serial.

python/å ä»£æ æ§å ¶ç¤ å″ /demo_can.py
CAN-style demo for the 4th-generation hand. It encodes addresses into extended identifiers, sends write and read frames, and reconstructs the returned values.

python/å ä»£æ æ§å ¶ç¤ å″ /demo_modbus.py
Controls one hand over Modbus TCP, defaulting to 192.168.11.210:6000. It writes angle, force, and speed registers, reads status, error, and temperature values, and runs an action sequence.

python/å ä»£æ æ§å ¶ç¤ å″ /demo_modbus_multi-device.py
Same Modbus approach, but for multiple hands at once. It tries to connect to several IPs, then starts one thread per device and runs the same control sequence in parallel.

python/å ä»£æ æ§å ¶ç¤ å″ /touch_data.py
Reads full tactile sensor register blocks over Modbus TCP. It formats raw data into matrices for pinky, ring, middle, index, thumb, and palm, then prints the structured tactile arrays.

python/å ä»£æ æ§å ¶ç¤ å″ /touch_data_ts.py
Reads higher-level tactile force values instead of full matrices. It extracts floating-point normal force and tangential force for each finger and prints them continuously.

C

C/control_485_C/hand_api.h
Header for the low-level RS-485 API. It defines protocol constants, data structures for finger state, and function declarations for reading and writing registers and controlling the hand.

C/control_485_C/hand_api.c
Implementation of the low-level RS-485 API. It builds serial frames, parses incoming frames, manages serial settings, and exposes functions like setting angle, speed, force, clearing errors, and reading status.

C/control_485_C/test.cpp
Small C++ test program that uses hand_api. It opens /dev/ttyUSB0, starts receive threads, and repeatedly runs Action(0) as a demo loop.

C/control_485_C/CMakeLists.txt
Build file for the C API and test program. It creates a static library

named hand_api and the test executable.

C/control_485_C/1.txt
A short usage note. It tells you to change the serial device and hand ID before building and running.

C++

C++/ä, ª å ä»£æ Ø ç ¤æ§å ¶ç¤ å” /control_485.cpp
Standalone C++ serial demo using Boost.Asio. It writes speed, force, and angle registers, reads back values, and prints raw frames and parsed results.

C++/ä, ª å ä»£æ Ø ç ¤æ§å ¶ç¤ å” /control_can.cpp
Standalone C++ CAN-style demo using Boost.Asio. It builds the custom frame format, handles timeouts, writes six-value register groups in two chunks, and reads them back.

Recommended starting points

Use python/.../demo_485.py for USB serial control.
Use python/.../demo_modbus.py for Ethernet or Modbus control.
Use touch_data.py or touch_data_ts.py if tactile sensor data is needed.